

BAHub - Phase 8: Project Management & RAID Logs

This document contains the project management structure, sprint plan, backlog, RAID log, and RACI matrix for the **BAHub** implementation.

1. Project Roadmap & Timeline

The implementation of BAHub spans a 16-week cycle divided into 4 core sprints:

```
[Sprint 1: Foundation (W1-4)] → [Sprint 2: Specs Grid (W5-8)] →  
[Sprint 3: Docs Sync (W9-12)] → [Sprint 4: AI & SaaS (W13-16)]
```

Key Project Milestones:

- **M-01 (Week 4):** Successful deployment of multi-tenant organization containers and JWT session auditing.
 - **M-02 (Week 8):** Verification of the split-pane requirements editor with sequential ID tracking.
 - **M-03 (Week 12):** Release of the document compiler (PDF/Word), signature validation workflows, and Jira sync.
 - **M-04 (Week 16):** Rollout of the AI Assistant, tenant subscription billing plans, and final UAT sign-off.
-

2. Sprint Backlogs

Sprint 1: Enterprise Multi-Tenant Foundation (Weeks 1–4)

- **Backlog Items:**
 1. Initialize Vite + React boilerplate and Django REST Framework environment.
 2. Implement `Organization` container model to enforce row-level tenant separation.
 3. Create custom `User` and `UserSession` models to log and audit user logins.
 4. Develop JWT authentication endpoints with automatic token-refresh interceptors.

- **Target Velocity:** 26 Story Points

Sprint 2: Collaborative Requirements Management (Weeks 5–8)

- **Backlog Items:**

1. Build the stakeholder directory and the interactive 2x2 Power/Interest matrix grid.
2. Build the Notion-style dual-pane requirements editor.
3. Implement Django ORM hooks to auto-generate sequential IDs (REQ-###) per project.
4. Integrate WebSocket endpoints (RequirementConsumer) to support real-time presence.

- **Target Velocity:** 28 Story Points

Sprint 3: Agile Boards, Document Compiling, & Sync (Weeks 9–12)

- **Backlog Items:**

1. Build the Agile board with Kanban lanes (*To Do, In Progress, Ready for QA, Done*) and story points.
2. Implement the BRD/FRD markdown document generation and signatory approval rules.
3. Integrate ReportLab and python-docx compilers to enable PDF and Word downloads.
4. Develop the Jira and Confluence integration panel, encrypting API keys with Fernet keys.

- **Target Velocity:** 32 Story Points

Sprint 4: Strategic Workspaces, AI, & SaaS Billing (Weeks 13–16)

- **Backlog Items:**

1. Develop SWOT quadrants and Gap Analysis spreadsheets.
2. Implement the AI Assistant chat interface with context-injection logic.
3. Integrate Stripe billing for plan subscriptions and seats limit checks.
4. Deploy to production environments (Vercel, Render, Neon PostgreSQL) and complete UAT.

- **Target Velocity:** 30 Story Points

3. RACI Matrix

This matrix maps project responsibilities across key roles:

- **R:** Responsible (performs the work)
- **A:** Accountable (directs and signs off)
- **C:** Consulted (provides input)
- **I:** Informed (kept updated)

Project Deliverable / Task	Business Analyst (BA)	Product Owner (PO)	Tech Lead / Architect	Lead QA Tester	Project Manager (PM)
Requirements Discovery & Mapping	R	A	C	I	I
Stakeholder Alignment Matrices	R	A	I	I	C
User Stories & Acceptance Criteria	R	A	C	C	I
Architecture Design & Database Schema	I	I	R	I	A
Jira / Confluence Sync Config	R	I	R	C	A
UAT Scenario Verification	C	A	I	R	I
SaaS Billing & Deployment Setup	I	I	R	I	A

4. RAID Log (Risks, Assumptions, Issues, Dependencies)

4.1 Risks (Identified Threats)

- **R-101: WebSocket Connection Drops**
 - *Description:* Collaborative editing sessions may drop WebSockets under unstable network environments.
 - *Impact:* Medium | *Probability:* High
 - *Mitigation:* Configure the frontend to auto-reconnect and implement visual connection status badges.
- **R-102: Third-Party API Key Exposure**
 - *Description:* Exposed Jira/Confluence integration keys could lead to security breaches.
 - *Impact:* High | *Probability:* Low
 - *Mitigation:* Encrypt API keys at rest in the database using Fernet symmetric keys.

4.2 Assumptions (Key Foundations)

- **A-101:** Users access the platform using modern web browsers that support the `wss://` WebSocket protocol.
- **A-102:** The Django backend is deployed on an ASGI server (such as Daphne) in production to support both WebSockets and traditional HTTP REST traffic.

4.3 Issues (Current Obstacles)

- **I-101: Local SQLite vs. Remote PostgreSQL Differences**
 - *Description:* SQLite does not enforce row-level constraints or schema migrations the same way PostgreSQL does.
 - *Impact:* Medium | *Status:* RESOLVED
 - *Resolution:* Configured Django settings to use `dj_database_url`. The system uses PostgreSQL when a `DATABASE_URL` environment variable is detected and defaults to SQLite locally.

4.4 Dependencies (Prerequisites)

- **D-101:** Integration with Jira requires the customer organization to have administrative access to configure project API integration keys.
- **D-102:** PDF document generation requires Python compilation libraries (such as WeasyPrint or ReportLab) to compile layout styling correctly.